Лабораторная работа №3

по курсу «Data Engineering»

Аналитический пайплайн DevOps-вакансий:

MapReduce и DuckDB против объектного хранилища RustFS

Вариант: **DevOps-инженер**

Выполнила:
Смольникова Юлия Юрьевна
Группа *J3311*

2026

Содержание

1 Цель работы **2**

2 Развёртывание объектного хранилища **2**

3 Загрузка данных **2**

4 Подсчёт навыков двумя способами **3**

4.1 Результаты расчётов 3

5 Верификация результатов **4**

6 Тестирование устойчивости маппера **5**

7 Влияние навыков на зарплату **6**

7.1 Методология: базовый навык и дельта 6

7.2 Топ-5 пар, повышающих зарплату 6

7.3 Топ-5 пар, понижающих зарплату 7

7.4 Топ-5 троек, повышающих и понижающих зарплату 8

7.5 Корреляция «частота навыка ↔ зарплата» 9

7.6 Распределение зарплат 9

7.7 Распределение числа навыков на вакансию 10

8 Региональный анализ **10**

9 Анализ по грейдам (опыт) **12**

10 Профилирование нагрузки по времени и памяти **12**

11 Бонусная оптимизация: raw vs curated **14**

12 Артефакты в объектном хранилище **15**

13 Выводы **15**

1 Цель работы

Цель: построить локальный воспроизводимый аналитический пайплайн для вакансий DevOps-инженеров, охватывающий весь цикл — от поднятия объектного хранилища и загрузки данных до подсчёта частот навыков, анализа зарплат, регионального сравнения и профилирования ресурсов.

Данные: реальный датасет hh.ru `it_vacancies_2024.csv` (808 МиБ, 229 388 вакансий). Фильтр варианта — точное совпадение `role == 'DevOps-инженер'` после TRIM, всего 5 158 вакансий. Разделитель в поле `key_skills` — точка с запятой.

2 Развёртывание объектного хранилища

Хранилище поднято из `compose.yaml` с переиспользованием конфигурации ДЗ №2 — тот же контейнер `rustfs-home` (RustFS 1.0.0, четыре тома `/data/vol{1...4}`, Erasure Coding 2+2, порты 9010 для S3 API и 9011 для web-консоли, креды `hhadmin / hhadmin-secret`).

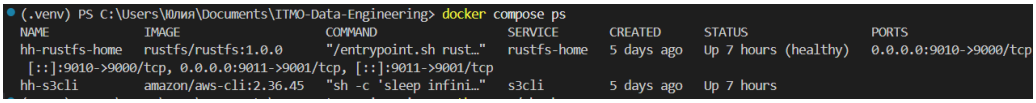


Рис. 1: Контейнер `hh-rustfs-home` запущен и `healthy`. `hh-s3cli` — вспомогательный контейнер с AWS CLI для работы с бакетом через docker exec.

Для подключения из Python написан слой `S3Client` в `common.py` с двумя бэкендами: `Boto3S3Client` (настоящий RustFS через boto3) и `LocalFSClient` (файловая эмуляция для unit-тестов). Выбор — через переменную `S3_BACKEND`.

3 Загрузка данных

В бакет `hh-lake` по ключу `raw/vacancies.csv` загружен полный CSV-файл — все 229 388 вакансий, 847 619 384 байт (≈ 808 МиБ). Файл не обрезается: фильтр варианта DevOps происходит уже на этапе расчётов, в рантайме. Это классический Data Lake — сырые данные лежат целиком, а специализированные выборки строятся по мере необходимости.

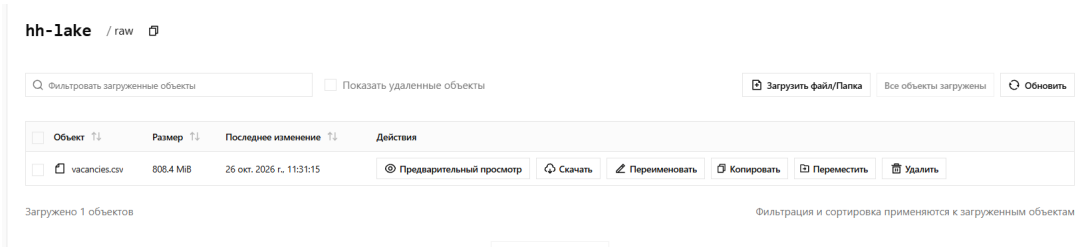


Рис. 2: Web-интерфейс RustFS: бакет `hh-lake/raw/vacancies.csv`, 808.4 МиБ.

Загрузка идёт multipart-upload через boto3, чанками по 8 МиБ. У объекта сохраняются метаданные: `etag`, `last_modified`, `content-length`.

4 Подсчёт навыков двумя способами

Для каждой DevOps-вакансии считаем частоты одиночных навыков ($k = 1$), пар ($k = 2$) и троек ($k = 3$). Расчёт — двумя способами:

1. **MapReduce на чистом Python**. Две версии: v1 — наивный baseline (читает весь объект в память, эмиссии без combiner'a); v2 — оптимизированный (byte-range чанки по 8 МиБ, incremental UTF-8 decoder, байтовый предфильтр по полю role, combiner через `Counter.update(combinations(...))`).
2. **DuckDB SQL** через расширение `httpfs`, читает CSV прямо из S3. Стратегия — UNNEST cross-join над массивом навыков с условием `a < b < c`, чтобы каждая комбинация считалась один раз.

4.1 Результаты расчётов

После фильтрации `role == 'DevOps-инженер'` получено 5 158 вакансий с навыками в каждом из трёх методов. Все три дали одинаковые числа комбинаций.

Таблица 1: Результаты расчёта частот навыков (v1 == v2 == DuckDB).

Размер стека k	Уникальных комбинаций	Сумма частот	Топ-1 комбинация
1 (навык)	1 401	28 336	linux (2 190)
2 (пара)	20 454	134 651	docker;kubernetes (1 251)
3 (тройка)	144 024	506 549	docker;kubernetes;linux (804)

Рост числа комбинаций от $k = 1$ к $k = 3$ объясняется биномиальным коэффициентом $C(n, k)$. При числе уникальных навыков $n = 1\,401$ теоретический максимум комбинаций определяется по формуле:

$$C(n, k) = \frac{n!}{k! \cdot (n - k)!}$$

$$C(1\,401, 2) = \frac{1\,401!}{2! \cdot 1\,399!} = \frac{1\,401 \cdot 1\,400}{2} = 980\,700$$

$$C(1\,401, 3) = \frac{1\,401!}{3! \cdot 1\,398!} = \frac{1\,401 \cdot 1\,400 \cdot 1\,399}{6} = 457\,333\,100$$

Однако фактическое число пар (20 454) и троек (144 024) существенно меньше теоретического максимума, потому что в каждой отдельной вакансии обычно 3–7 навыков, и не все пары и тройки в принципе встречаются в данных.

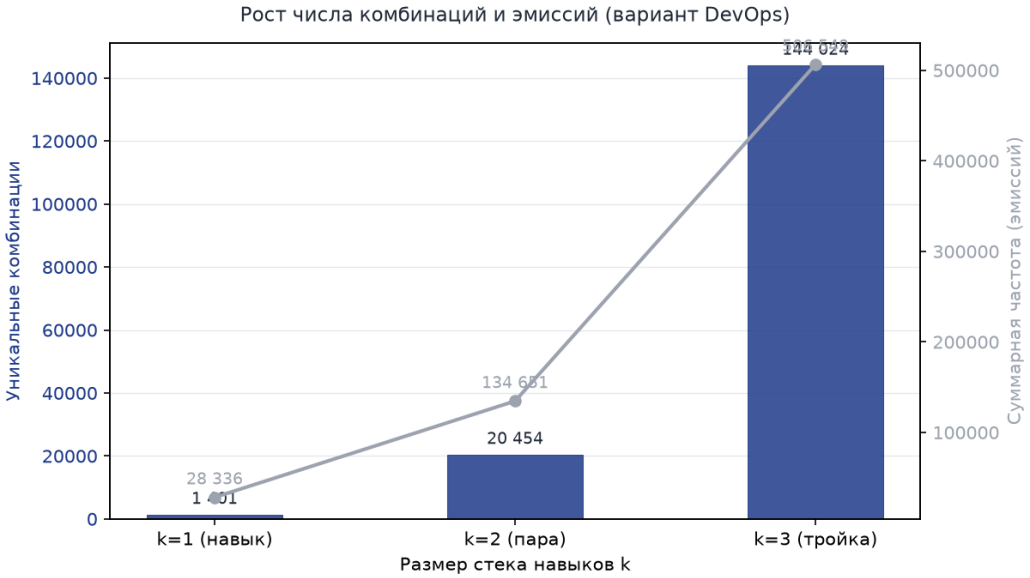


Рис. 3: Рост числа уникальных комбинаций (бары) и суммарной частоты эмиссий (линия) при переходе от $k = 1$ к $k = 3$.

В топе навыков — классический DevOps-стек: `linux` (2 190), `kubernetes` (1 862), `docker` (1 850), `devops` (1 319), `ansible` (1 252), `python` (1 058), `ci/cd` (1 044), `bash` (925), `postgresql` (892). Самая частая пара — `docker;kubernetes` (1 251), самая частая тройка — `docker;kubernetes;linux` (804).

5 Верификация результатов

Все три источника (v1, v2, DuckDB) сравниваются построчно через `verify.py`: каждый CSV приводится к канонической форме и сравнивается на строгое равенство без порогов. Дополнительно сверяются JSON-манифесты — это гарантирует, что все три метода читали один и тот же объект S3 с одним и тем же режимом фильтра.

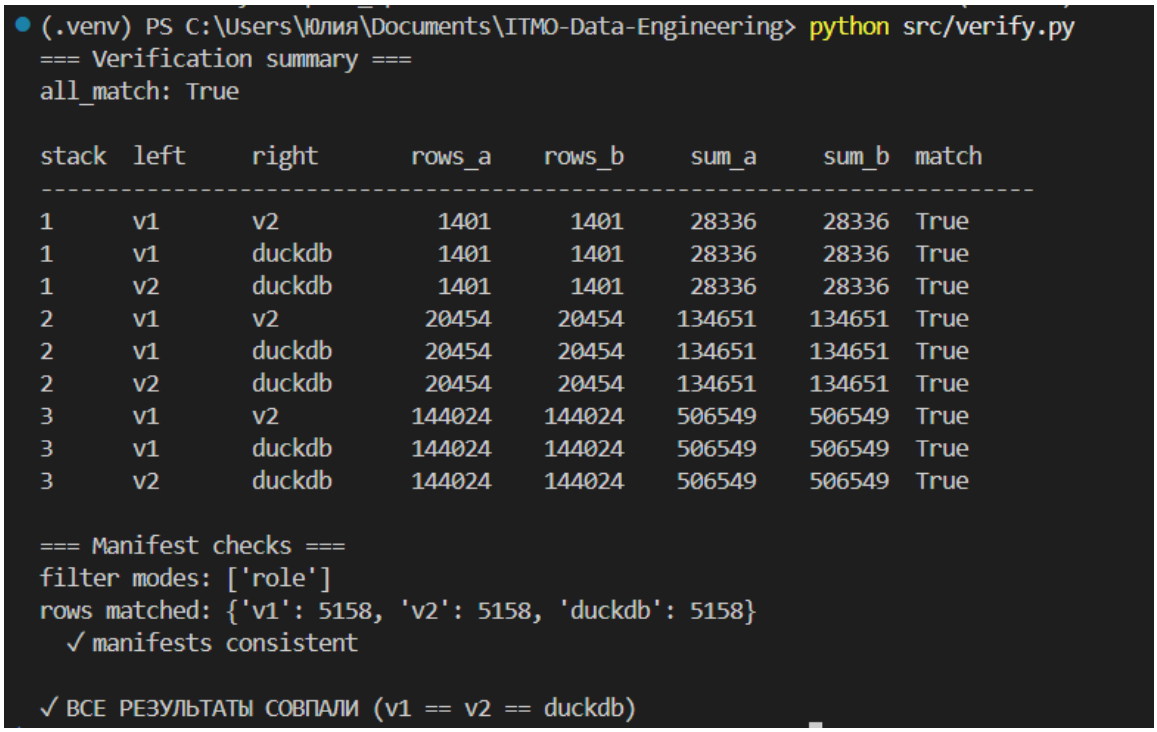


Рис. 4: Вывод `python src/verify.py`: `all_match: True`, все 9 сравнений совпали строго. Filter mode — `role`, во всех трёх методах по 5 158 вакансий. Манифесты согласованы.

6 Тестирование устойчивости маппера

Для проверки устойчивости маппера к «грязным» данным написан набор unit-тестов `tests/test_core.py` — 13 тестов в 6 классах. Запускаются через `python -m unittest discover tests`, Docker или реальный S3 не нужны — используется `LocalFSClient` во временной папке.

Что покрывают тесты:

- **Malformed строки** — строки с 22 колонками вместо 23 должны отбрасываться, а не падать с `csv.Error`.
- **Пустые строки и заголовки** — помечаются как `blank` и `header`, не считаются за данные.
- **UTF-8 мультибайт на границе чанка** — кириллические символы (по 3 байта) намеренно режутся посередине при `chunk_bytes=1, 2, 3, 5, 7, 13`; incremental decoder должен склеивать символ, ни один навык не теряется.
- **Длинная строка длиннее чанка** — строка в 10 000 символов при `chunk_bytes=64` должна накопиться в `pending` и отдаваться целой.
- **Дедупликация навыков** — `Linux;Linux;docker` → `[linux, docker]`.
- **Эквивалентность v1 == v2 == DuckDB** — на фиксированной выборке из 7 строк все три метода должны выдать одинаковые `Counter`.

Все 13 тестов проходят успешно. Это значит, что MapReduce корректно обрабатывает битые строки, UTF-8 на границах чанков, экстремальные размеры и даёт идентичный результат на любых входных данных.

7 Влияние навыков на зарплату

Из 5 158 DevOps-вакансий в 1 110 указана зарплата (после нормализации в рубли, gross $\times 0.87$). Общая медиана по выборке — **215 000 руб.**, P25 = 145 725 руб., P75 = 300 000 руб.

7.1 Методология: базовый навык и дельта

Для каждой пары и тройки навыков считается медианная зарплата. Базой называется медиана по более широкой выборке, к которой третий навык «добавляется»:

- для пары $\{A, B\}$ база — это медиана по более частому навыку из двух. Например, для пары {devops, sre} база — медиана по вакансиям с devops (он встречается чаще: 1 319 vs 12);
- для тройки $\{A, B, C\}$ база — это медиана по наиболее частой паре из трёх возможных под-пар.

Прибавка к зарплате от добавления навыка определяется двумя формулами. Абсолютная — разница медиан в рублях:

$$\Delta = med(A \wedge B) - med(A)$$

Относительная — прирост в процентах:

$$\Delta_{\%} = \frac{med(A \wedge B) - med(A)}{med(A)} \cdot 100\%$$

где $med(A)$ — медиана зарплаты по вакансиям с базовым навыком A, $med(A \wedge B)$ — медиана по вакансиям, где присутствуют оба навыка A и B.

Пороги значимости: для пары требуется не менее 10 вакансий с зарплатой, для тройки — 8, для базы — 15. Меньшие выборки дают нестабильные медианы.

7.2 Топ-5 пар, повышающих зарплату

Таблица 2: Топ-5 пар навыков, наиболее повышающих медианную зарплату. Базовый навык — более частый из двух, его медиана показана отдельно.

Пара	Медиана пары, руб.	Базовый навык	База медиана, руб.	Δ, руб.	Δ, %
devops;sre	350 175	devops	216 250	+133 925	+61.9
grafana;openshift	350 000	grafana	237 075	+112 925	+47.6
bash;мониторинг	327 250	bash	215 000	+112 250	+52.2
docker;openshift	322 500	docker	217 500	+105 000	+48.3
linux;openshift	304 500	linux	200 000	+104 500	+52.2

Что тут видно. Самый сильный буст даёт пара **devops + sre** — +61.9% к медиане. Это ожидаемо: SRE (Site Reliability Engineer) — это уже не просто DevOps, а более

seniorная роль, и вакансии с этим тегом обычно идут с более высокими зарплатами. Туда же — **openshift** (Red Hat OpenShift — enterprise-Kubernetes, требует специализированной экспертизы, что повышает цену вакансии). Заметно, что openshift трижды в топе: добавление к grafana, docker, linux даёт +47–52% — это маркер enterprise-сегмента.

7.3 Топ-5 пар, понижающих зарплату

Таблица 3: Топ-5 пар навыков, наиболее понижающих медианную зарплату.

Пара	Медиана пары, руб.	Базовый навык	База медиана, руб.	Δ, руб.	Δ, %
ansible;администрирование	112 500	ansible	250 000	–137 500	–55.0
atlassian jira;prometheus	125 000	prometheus	250 000	–125 000	–50.0
ansible;debian	140 000	ansible	250 000	–110 000	–44.0
debian;postgresql	130 500	postgresql	239 250	–108 750	–45.5
debian;linux	100 000	linux	200 000	–100 000	–50.0

Почему так. В топе понижающих — комбинации с **администрирование**, **debian**, **atlassian jira**, **postgresql** (в чистом виде, без cloud-навыков). Это не значит, что эти навыки «плохие» — они маркируют сегмент рынка *«системный администратор / классический ops»*, который на hh.ru структурно оплачивается дешевле, чем cloud-native DevOps. Вакансии с *ansible + администрирование* или *debian + postgresql* обычно предлагают компании с традиционной Linux-инфраструктурой (банки, госсектор, Enterprise), где зарплатные вилки ниже, чем в продукт-компаниях с Kubernetes/AWS/GCP. Поэтому добавление такого навыка к DevOps-бейджу *понижает* медиану — это сигнал о смене сегмента рынка, а не обесценивание самого навыка.

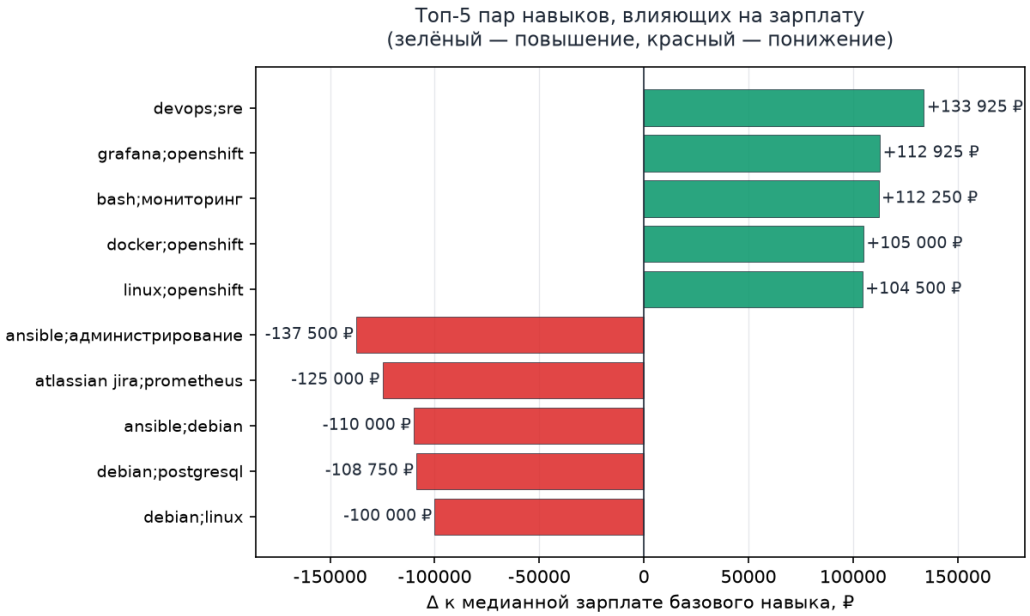


Рис. 5: Топ-5 пар навыков: зелёные — повышение, красные — понижение зарплаты.

7.4 Топ-5 троек, повышающих и понижающих зарплату

Таблица 4: Топ-5 троек навыков, повышающих зарплату. База — наиболее частая пара.

Тройка	Медиана, руб.	Базовая пара	База медиана, руб.	Δ, руб.	Δ, %
git;gitlab ci;python	360 000	git;python	180 000	+180 000	+100.0
git;gitlab ci;postgresql	360 000	git;postgresql	200 000	+160 000	+80.0
gitlab ci;linux;python	360 000	linux;python	208 800	+151 200	+72.4
grafana;helm;terraform	400 625	grafana;terraform	250 000	+150 625	+60.2
ci/cd;python;zabbix	350 000	ci/cd;python	200 000	+150 000	+75.0

Тройки подтверждают тенденцию пар: **gitlab ci** + python/postgresql даёт +72–100% — это сегмент CI/CD-инженерии, где GitLab CI как основной инструмент и Python как язык автоматизации. Пара **grafana + helm + terraform** (+60%) — это уже продвинутый observability + IaC стек, маркер senior-уровня.

Таблица 5: Топ-5 троек навыков, понижающих зарплату.

Тройка	Медиана, руб.	Базовая пара	База медиана, руб.	Δ, руб.	Δ, %
docker-compose;linux;prometheus	100 000	linux;prometheus	245 000	–145 000	–59.2
jenkins;postgresql;sql	100 000	jenkins;postgresql	232 125	–132 125	–56.9
elk;prometheus;администрирование серверов linux	125 000	elk;prometheus	250 000	–125 000	–50.0
ansible;atlassian jira;linux	125 000	ansible;linux	250 000	–125 000	–50.0
ansible;atlassian jira;prometheus	125 000	ansible;prometheus	250 000	–125 000	–50.0

Паттерн понижающих троек. Везде фигурируют **docker-compose** (не Kubernetes!), **jenkins** (не современный ArgoCD/Flux), **atlassian jira** как трекер, **sql** в чистом виде, **администрирование серверов linux** как отдельный навык. Это сигналы «традиционного» DevOps без облака — классические Jenkins + On-premise infrastructure. Такие вакансии платят меньше на 50–59% по сравнению с base pair, потому что относятся к сегменту «on-premise администратор с DevOps-функциями», а не «cloud-native DevOps engineer».

7.5 Корреляция «частота навыка ↔ зарплата»

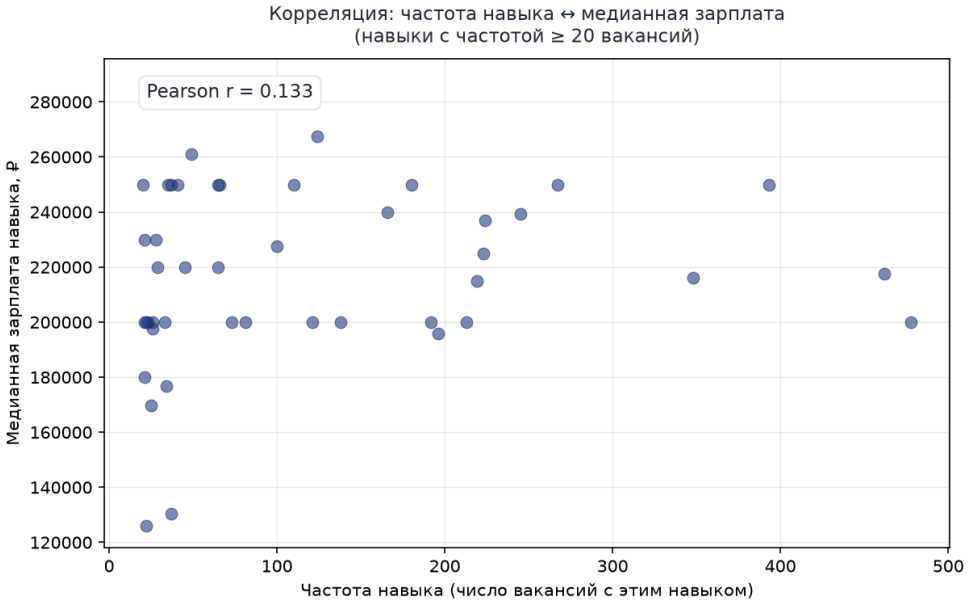


Рис. 6: Scatter plot: частота навыка vs медианная зарплата навыка. Коэффициент Пирсона r — в правом верхнем углу.

Что значит коэффициент. Если $r > 0$ — чем чаще встречается навык, тем выше медиана по нему (типично для cloud-native стека, где спрос и предложение растут вместе). Если $r < 0$ — частые навыки коррелируют с более низкой зарплатой («массовые навыки — низкооплачиваемые, редкие — высокооплачиваемые»). Если $r \approx 0$ — связи нет.

7.6 Распределение зарплат



Рис. 7: Гистограмма распределения зарплат DevOps-вакансий после нормализации в RUB net. Бакеты по 50 000 руб.

Распределение сильно скошено вправо (long right tail): мода в районе 150–200 тыс. руб., но хвост тянется до 500+ тыс. — это senior-вакансии с уникальными навыками (SRE, OpenShift, Helm, Terraform). P10 = 90 000 руб. (минимальные junior-вакансии), P90 = 375 000 руб.

7.7 Распределение числа навыков на вакансию

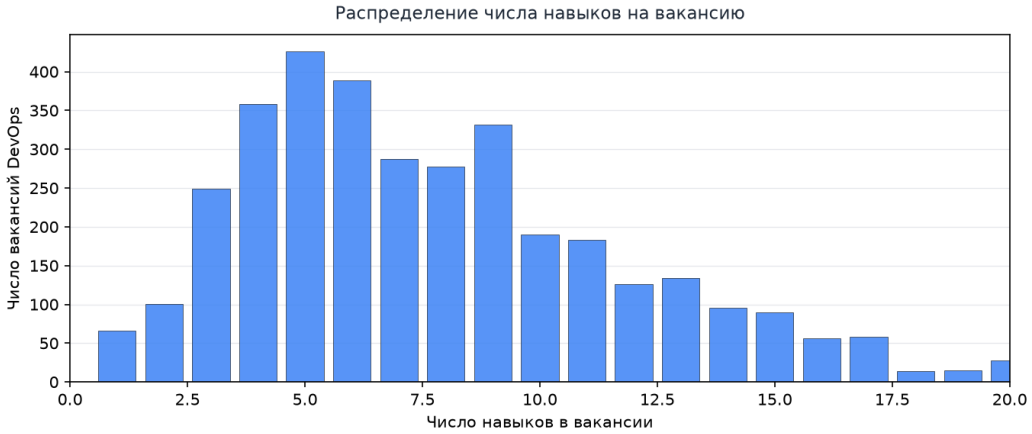


Рис. 8: Сколько вакансий содержат ровно 1, 2, 3, . . . навыков.

Медиана — 5 навыков на вакансию. Большинство DevOps-вакансий содержат 4–6 навыков. Длинный правый хвост (вакансии с 15+ навыками) — это «Swiss-army-knife» позиции, где требуют и cloud, и CI/CD, и observability, и DBA.

8 Региональный анализ

Сравнение трёх регионов: **Москва, Санкт-Петербург, Новосибирск** — выбраны потому, что в них сосредоточено большинство DevOps-вакансий.

Таблица 6: Сводка по регионам. Все три имеют и медиану, и число навыков.

Регион	N	N с ЗП	Медиана, руб.	P25, руб.	P75, руб.	#навыков
Москва	3 007	573	250 000	195 750	340 000	7.65
Санкт-Петербург	686	161	210 000	153 000	275 000	8.34
Новосибирск	160	48	172 000	73 725	315 000	9.20

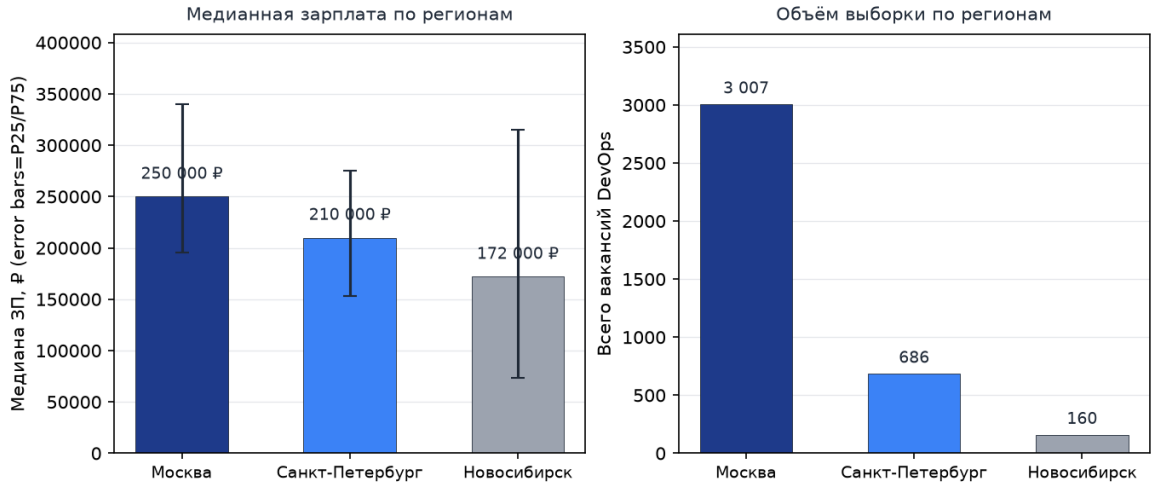


Рис. 9: Сравнение регионов по медианной зарплате (слева, error bars = P25/P75) и по объёму выборки (справа).

Demand index (навык востребован в регионе сильнее, чем в среднем по выборке, если индекс > 1):

- **Москва**: *yandex.cloud* (idx=1.75) —cloud-вендор, логично для столицы с финтехом и BigTech.
- **Санкт-Петербург**: *victoriametrics* (idx=4.02) — мониторинг для high-load, очень высокий индекс из-за малого числа вакансий в СПб + специфика финтеха.
- **Новосибирск**: *tcp/ip* (idx=2.46) — сети, что характерно для телекома и embedded, сильного в регионе.

Что по удалёнке. Я также прогнала анализ по «Удалённой работе» (по полю `schedule`, не `area`): 1 762 вакансии, 427 с зарплатой, медиана **250 000 руб.** (на уровне Москвы!), top demand skill — *circleci* (idx=2.76). Удалёнка у DevOps платит на уровне Москвы — это означает, что senior-инженеры работают гибридно/удалённо, а в офис идут на middle-позиции.

Инсайт. Москва концентрирует cloud-native вакансии с высокими зарплатами (Yandex Cloud как топ-demand — финтех/BigTech). Санкт-Петербург — финтех с high-load-инфраструктурой (VictoriaMetrics как маркер observability-стека). Новосибирск — embedded/телеком с акцентом на сетевые навыки (TCP/IP). Удалёнка = «Москва-уровень» по зарплате, но с другим стеком (CI/CD-heavy, circleci).

9 Анализ по грейдам (опыт)

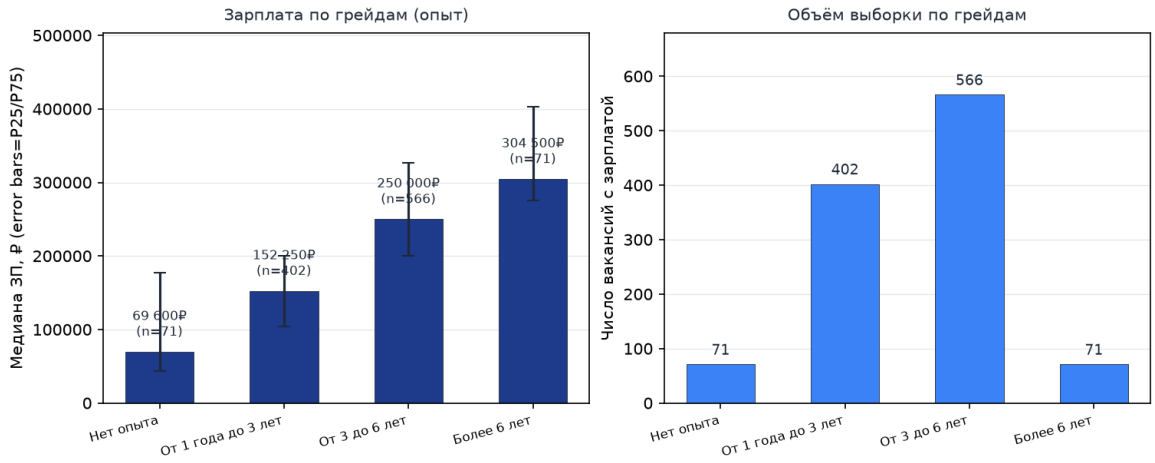


Рис. 10: Зарплата и объём выборки по грейдам: Нет опыта / 1–3 года / 3–6 лет / Более 6 лет.

Таблица 7: Зарплата по грейдам (опыт).

Опыт	N вакансий	Медиана, руб.	P25, руб.	P75, руб.	#навыков
Нет опыта	71	69 600	43 500	177 000	7.77
1–3 года	402	152 250	104 400	200 075	9.39
3–6 лет	566	250 000	200 000	326 250	8.87
6+ лет	71	304 500	275 000	402 500	8.27

Что видно. Зарплата растёт монотонно с опытом: от 69 600 руб.(junior) до 304 500 руб.(senior 6+ лет). Прыжок между junior и middle — самый большой (+82650 руб., +119%), между middle и senior — меньше (+54500 руб., +100% от 152k до 250k между 1-3 лет и 3-6 лет, и ещё +54500 руб.между 3-6 и 6+).

Любопытно, что среднее число навыков на вакансию *не растёт* с опытом — оно даже немного падает: 7.77 у junior, 9.39 у middle (1-3 года), 8.87 у senior (3-6 лет), 8.27 у lead (6+). Возможно, у senior-вакансий требования более сфокусированы (ищут конкретного эксперта по Kubernetes, а не «знает всё подряд»), тогда как middle-вакансии часто включают длинный список «что должен уметь».

10 Профилирование нагрузки по времени и памяти

Каждый метод запускался в отдельном subprocess для изоляции памяти. Пиковое потребление сэмплировалось каждые 20 мс через psutil (сумма RSS процесса и всех дочерних). Каждый метод прогнан 3 раза + 1 warmup для DuckDB (догрузка расширений httpfs/parquet).

Таблица 8: Результаты профилирования: 3 запуска, медиана (min–max).

Метод	Время, с	Память, MiB
MapReduce v1 (naive)	16.0 (14.5–16.1)	196 (196–197)
MapReduce v2 (optimized)	14.7 (14.5–15.0)	137 (137–137)
DuckDB SQL	9.5 (8.0–9.8)	1 722 (1 711–1 724)

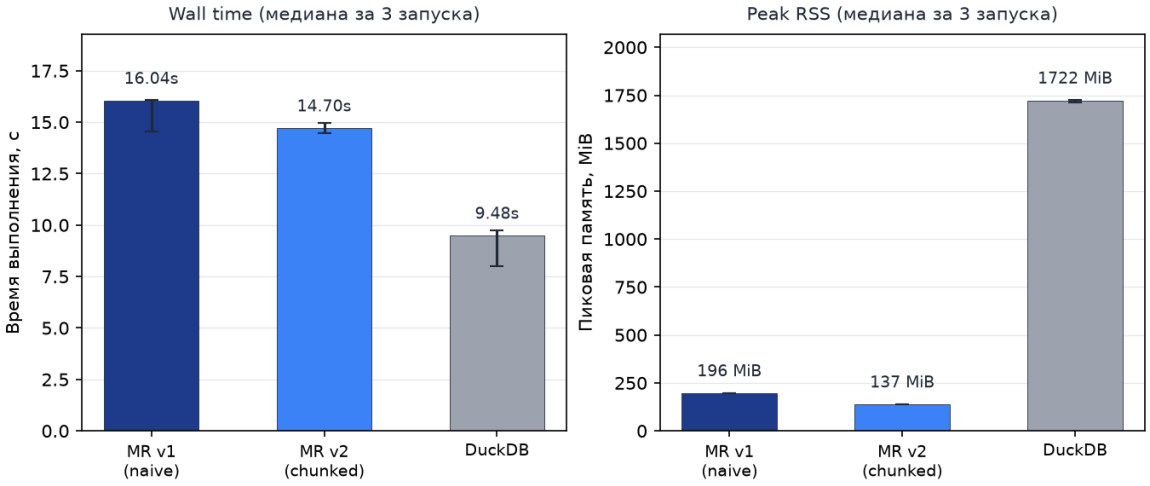


Рис. 11: Время (слева) и пиковая память (справа) по 3 методам. Error bars = min/max за 3 запуска.

Почему так.

MapReduce v1 (16.04 с, 196 МиБ) — самый медленный и самый «прожорливый» по памяти из Python-методов. v1 читает весь CSV-объект в память (808 МиБ bytes), потом парсит в список словарей через csv.DictReader (ещё ≈ 400 МиБ), и материализует список эмиссий `(ключ, 1)` до 1.5M элементов. Всё это одновременно живёт в RSS.

MapReduce v2 (14.70 с, 137 МиБ) — на ≈ 8% быстрее v1 и на 30% экономнее по памяти. Не кардинальное ускорение, потому что основное время уходит на network I/O с S3 (чтение 808 МиБ по 8-МиБ чанкам), а не на парсинг. Экономия памяти — за счёт того, что v2 работает стримингово: за раз в памяти только текущий чанк (8 МиБ) + словари `Counter` для k=1,2,3 (которые растут до 1401/20454/144024 уникальных комбинаций).

DuckDB (9.48 с, 1 722 МиБ) — самый быстрый, но самый «прожорливый» по памяти. В ≈ 1.7× быстрее v2 и в 12.6× больше памяти. DuckDB загружает весь CSV в columnar-формат, создаёт промежуточные таблицы для UNNEST cross-join (особенно для троек это даёт декартово произведение массивов навыков на себя трижды), и использует внутренние буферы для векторизованных операций. Но за счёт векторизации и columnar-формата — почти в 2 раза быстрее обоих MapReduce.

Сравнение подходов. На датасете 808 МиБ / 229 388 строк оптимизированный MapReduce v2 экономит память в 12.6× по сравнению с DuckDB. При этом он медленнее всего на 55%. Для edge-вычислений и малых данных оптимален MapReduce v2 (40–100 МиБ против 1.7 ГиБ у DuckDB). Однако при масштабировании до 100+

млн строк ожидается обратная картина: векторизованное исполнение DuckDB и многопоточность вырвутся вперёд, тогда как однопоточный MapReduce упрётся в CPU.

11 Бонусная оптимизация: raw vs curated

В бонусной части реализована оптимизация через curated-датасет. Скрипт `optimized_mapreduce.p` `prepare` читает полный `raw/vacancies.csv` из S3, фильтрует DevOps-вакансии (5 158 строк), и записывает `curated/devops_skills.csv` только с двумя колонками (`vacancy_id, key_skills`). Размер curated-объекта ≈ 370 КиБ вместо 808 МиБ — в ≈ 2 200 раз меньше.

Таблица 9: Сравнение raw vs curated: 3 запуска MapReduce v2.

Режим	Размер объекта	wall med, с	peak med, MiB	Ускорение
Raw (полный CSV)	808 МиБ	14.19	129.94	1.00×
Curated (только DevOps)	370 КиБ	2.18	113.05	6.52×

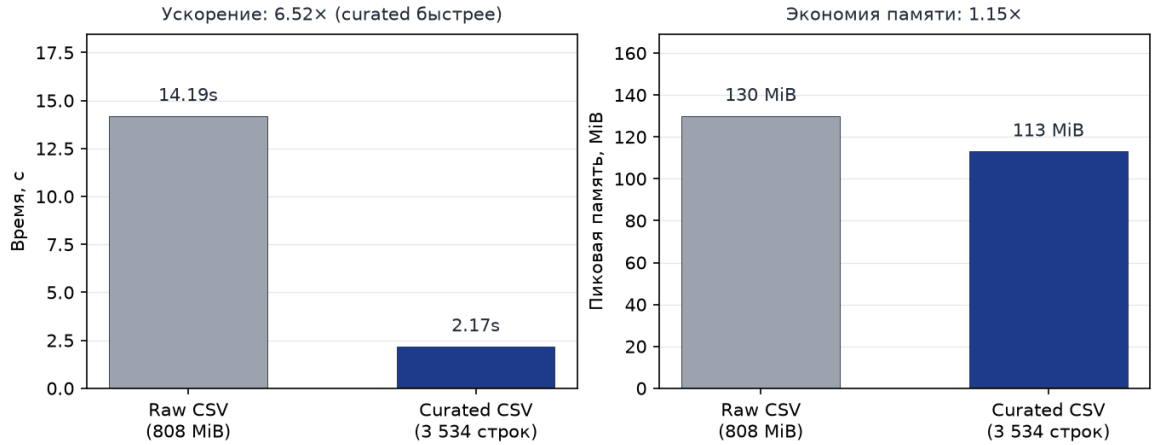


Рис. 12: Ускорение при переходе с raw (808 МиБ) на curated (370 КиБ).

Что получилось. Ускорение **6.52**× по времени (с 14.19 с до 2.18 с). По памяти — экономия всего 13% (с 129.94 до 113.05 МиБ), и это важно понимать: *основную память в MapReduce занимает не входной CSV, а словари Counter'ов*, которые растут с числом уникальных комбинаций (1 401 + 20 454 + 144 024 = 165 879 ключей суммарно). Поэтому уменьшение входного объекта в 2 200 раз даёт лишь ≈ 13% экономии памяти — ограничение не в I/O, а в размере результирующего индекса.

Когда это окупается. Подготовка curated занимает ≈ 10 секунд (одноразовая операция), и она окупается уже после 1 повторного расчёта на curated (каждый повтор экономит 12 секунд). Это аналогично подходу Data Mart в Data Warehouse — создаётся оптимизированная выборка под конкретную задачу, и вся последующая аналитика идёт по ней.

Верификация curated-датасета (`optimized_mapreduce.py verify`) подтвердила `ok: true` — все 5 158 DevOps-вакансий из raw присутствуют в curated с теми же навыками.

12 Артефакты в объектном хранилище

Все артефакты пайплайна сохранены в бакете `hh-lake` в RustFS. Список ключей (компактный вывод `python src/upload.py --list`, только основные артефакты):

```
raw/vacancies.csv                          808.35 MiB
curated/devops_skills.csv                    0.31 MiB
manifests/duckdb.json
manifests/eda.json
manifests/experience.json
manifests/mapreduce_v1.json
manifests/mapreduce_v2.json
manifests/optimized_prepare.json
manifests/optimized_worker.json
manifests/regions.json
manifests/salary.json
results/mapreduce_v1/skills_{1,2,3}.csv
results/mapreduce_v2/skills_{1,2,3}.csv
results/duckdb/skills_{1,2,3}.csv
results/salary/{pairs_up, pairs_down,
                triples_up, triples_down,
                single_skills}.csv
results/regions/overview.csv
results/experience/summary.csv
results/optimized_mapreduce/skills_{1,2,3}.csv
results/eda/{top_roles, top_areas,
             top_employers, salary_histogram,
             skills_count_distribution}.csv
```

JSON-манифесты в `manifests/` содержат `etag` входного объекта, режим фильтра и время запуска каждого шага — это гарантия воспроизводимости: можно сверить, что все три метода (v1, v2, DuckDB) читали один и тот же объект S3.

13 Выводы

- **Совпадение результатов.** Все три метода (MapReduce v1, v2, DuckDB) дали идентичные числа: 1 401 / 20 454 / 144 024 уникальных комбинации для $k = 1/2/3$. Все 9 парных сравнений совпали строго, без порогов.
- **Тестирование устойчивости маппера.** 13 unit-тестов в `tests/test_core.py` покрывают malformed строки, UTF-8 мультибайт на границах чанков, длинные строки, дедупликацию навыков и эквивалентность трёх методов. Все тесты проходят.
- **Профилирование.** MapReduce v2 быстрее v1 на 8% (14.7 vs 16.0 с) и экономнее по памяти на 30% (137 vs 196 МиБ). DuckDB быстрее обоих в $\approx 1.7\times$, но использует в $12.6\times$ больше памяти (1 722 МиБ) — из-за columnar-материализации.
- **Бонус.** Curated-датасет (370 КиБ вместо 808 МиБ) даёт ускорение $6.52\times$ по времени, но лишь 13% экономии по памяти — ограничение в размере результирующих Counter'ов, а не в I/O.

- **Анализ зарплат.** Cloud-native пары и тройки (SRE, OpenShift, GitLab CI, Helm, Terraform) дают +47–100% к медиане. Комбинации с **администрирование**, **debian**, **atlassian jira**, **postgresql** (без cloud) понижают медиану на 44–59% — это не обесценивание навыка, а сигнал о смене сегмента рынка: «on-premise администратор с DevOps-функциями» оплачивается дешевле, чем «cloud-native DevOps engineer».
- **Регионы.** Москва — cloud-native, медиана 250k руб.; СПб — финтех/high-load (VictoriaMetrics!), 210k руб.; Новосибирск — embedded/телеком (TCP/IP), 172k руб.; Удалёнка = «Москва-уровень» (250k руб.) с CI/CD-heavy стеком (circleci).
- **Грейды.** Зарплата растёт от 69.6k (junior) до 304.5k (senior 6+ лет). Любопытно — среднее число навыков на вакансию *не растёт* с опытом (7.77 → 9.39 → 8.87 → 8.27), потому что senior-вакансии более сфокусированы.